Frontend Testing Automation Platform - Development Specification

Purpose:

Build a reusable automated testing web application for React/frontend projects.
The system should analyze an existing project, identify current testing coverage,
detect missing parts, and generate/manage automated tests.

Core Objective:

Given an existing frontend codebase, the platform should inspect:

- Components
- Pages
- Routes
- Hooks
- API integrations
- State management
- User flows
- Existing tests
- Storybook stories

Then generate a testing gap report and development plan.

Testing Areas Required:

1. Component Testing

- Component rendering
- Props validation
- Conditional rendering
- UI states:
- Loading
- Empty
- Error
- Success

2. User Interaction Testing

- Click events
- Input changes
- Form interactions
- Keyboard navigation
- Dropdowns
- Modals
- Tabs
- Drag and drop

3. Form Testing

- Required fields
- Validation rules
- Error messages
- Submission flows
- File uploads

4. Accessibility Testing

- ARIA validation
- Keyboard navigation
- Screen reader support
- Focus handling
- Color contrast checks

5. Visual Regression Testing

- Screenshot comparison
- UI breaking detection
- Component visual states

6. Responsive Testing

Test:

- Mobile
- Tablet
- Desktop
- Different screen sizes

7. API Integration Testing

- GET/POST/PUT/DELETE flows
- Loading handling
- Error handling
- Mock API support

8. State Management Testing

Support:

- Redux
- Zustand
- Context API
- Other stores

9. Routing Testing

- Route validation

- Protected routes
- Redirect handling
- 404 pages

10. Authentication Testing
- Login
- Logout
- Token handling
- Permission testing
- Role based access

11. Table/Data Testing
- Search
- Filter
- Sorting
- Pagination
- Export
- Selection

12. Performance Testing
- Render performance
- Large data handling
- Bundle analysis
- Memory issues

13. Browser Testing
Support:
- Chrome
- Firefox
- Safari
- Edge

14. Security Testing
- XSS checks
- Unsafe HTML usage
- Sensitive data exposure

Storybook Requirements:

Every component should have generated stories:

States:
- Default

- Loading
- Disabled
- Error
- Empty
- Dark mode
- Mobile view
- Long content

Testing Stack:

Unit and Component:
- Vitest
- React Testing Library

Mocking:
- MSW

BDD:
- Cucumber.js

Browser Automation:
- Playwright

Visual Testing:
- Storybook + Screenshot comparison

Accessibility:
- axe-core

Platform Features:

1. Project Scanner
- Detect framework
- Detect components
- Detect existing tests
- Detect missing coverage

2. Test Coverage Dashboard
Show:
- Tested components
- Untested components
- Missing scenarios
- Coverage percentage

3. Test Generator

Generate:

- Vitest tests
- Storybook stories
- Cucumber scenarios
- Playwright flows

4. AI Analysis Module

Analyze:

- Component purpose
- Expected behavior
- Missing tests
- Suggested test cases

5. CI/CD Integration

Support:

- GitHub Actions
- Automated test execution
- Test reports

Development Task:

First analyze the existing application structure.

Do not rebuild blindly.

Steps:

1. Understand current architecture
2. Identify implemented modules
3. Identify missing modules
4. Create development roadmap
5. Implement missing features
6. Improve existing code quality
7. Add automated tests for the platform itself

Final Goal:

A universal frontend testing automation platform that can be connected to any React/Next.js project and automatically create, analyze, and maintain frontend testing workflows.